

Creating a Confidence Ellipses in a sccatterplot using matplotlib

Asked 7 years, 4 months ago Active 2 months ago Viewed 23k times

How to creating a Confidence Ellipses in a sccatterplot using matplotlib?

22 The following code works until creating scatter plot. Then, does anyone familiar with putting Confidence Ellipses over the scatter plot?

```
import numpy as np
import matplotlib.pyplot as plt
x = [5,7,11,15,16,17,18]
y = [8, 5, 8, 9, 17, 18, 25]

plt.scatter(x,y)
plt.show()
```

Following is the reference for Confidence Ellipses from SAS.

<http://support.sas.com/documentation/cdl/en/grstatproc/62603/HTML/default/viewer.htm#a003160800.htm>

The code in sas is like this:

```
proc sgscatter data=sashelp.iris(where=(species="Versicolor"));
  title "Versicolor Length and Width";
  compare y=(sepalwidth petalwidth)
          x=(sepallength petallength)
          / reg ellipse=(type=mean) spacing=4;
run;
```

python numpy matplotlib scipy Edit tags

Share Edit Follow Close Flag Protect

edited Nov 21 '13 at 16:41

asked Nov 21 '13 at 16:09

 2964502

3,445 7 28 52

- 1 possible duplicate of [multidimensional confidence intervals](#) – Saullo G. P. Castro Nov 21 '13 at 16:35
- 2 @ Saullo Castro have you seen the code in sas and do you think that the method implemented in sas and in the link you provided the same? – 2964502 Nov 21 '13 at 16:42
- 2 @tester3 - In the example you linked to, the confidence ellipse shown is for the mean, as opposed to for another sample drawn from the same population. (This is what `type=mean` is specifying.) My answer that @SaulloCastro linked to shows a confidence ellipse for the entire population (in other words, the area that another sample from the population should fall inside, identical to `type=predicted` in SAS). Jamie's answer uses this method as well. – Joe Kington Nov 21 '13 at 23:35

3 Answers

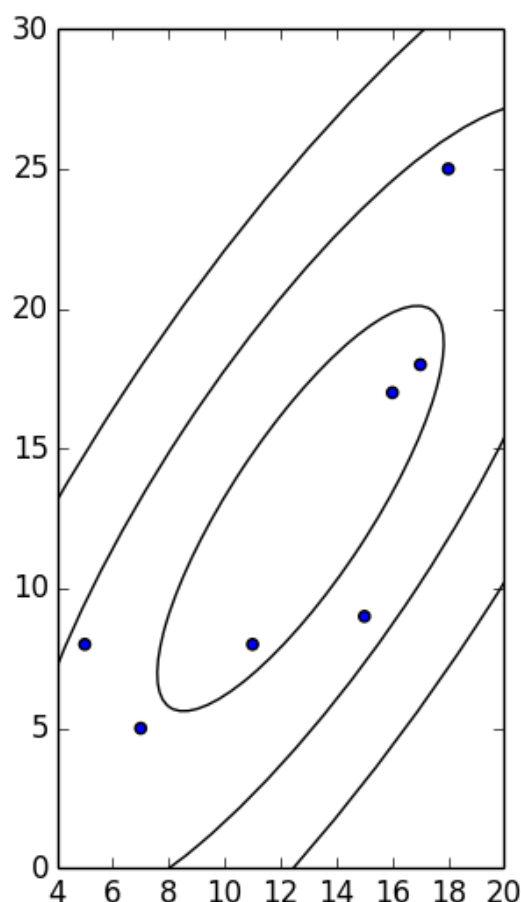
Active	Oldest	Votes
--------	--------	-------

The following code draws a one, two, and three standard deviation sized ellipses:

27



```
x = [5,7,11,15,16,17,18]
y = [8, 5, 8, 9, 17, 18, 25]
cov = np.cov(x, y)
lambda_, v = np.linalg.eig(cov)
lambda_ = np.sqrt(lambda_)
from matplotlib.patches import Ellipse
import matplotlib.pyplot as plt
ax = plt.subplot(111, aspect='equal')
for j in xrange(1, 4):
    ell = Ellipse(xy=(np.mean(x), np.mean(y)),
                  width=lambda_[0]*j*2, height=lambda_[1]*j*2,
                  angle=np.rad2deg(np.arccos(v[0, 0])))
    ell.set_facecolor('none')
    ax.add_artist(ell)
plt.scatter(x, y)
plt.show()
```



Share Edit Follow Flag

edited Nov 21 '13 at 19:55

answered Nov 21 '13 at 17:04



Jaime

58.5k

13

105

147

1 ▲ @Jamie - +1 Shouldn't the ellipses be twice as wide and high, though? Currently, they're N-sigma wide and high, as opposed to showing the region within N-sigma of the mean. – [Joe Kington](#) Nov 21 '13 at 19:18

▲ @JoeKington Yes, I do think you are absolutely right, matplotlib makes it kind of clear they are the width and height, not the semi-width and semi-height... Have edited the code and image. Thanks! – [Jaime](#) Nov 21 '13 at 19:55

1 ▲ Better check @Ben 's answer below, because this code doesn't compute the angle properly. It appeared about 90 degrees flipped in my case. – [grasshopper](#) May 11 '16 at 15:20

4 ▲ Can confirm that this example computes the angle incorrectly; the correct angle is computed as `angle=np.rad2deg(np.arctan2(*v[:,0][::-1]))` – [nathan lachenmyer](#) Jan 28 '20 at 19:24

▲ *In addition to the accepted answer:* I think the correct angle should be:

0

`angle=np.rad2deg(np.arctan2(*v[:,np.argmax(abs(lambda_))][::-1]))`

▼ and the corresponding width (larger eigenvalue) and height should be:



```
width=lambda_[np.argmax(abs(lambda_))*j*2, height=lambda_[1-  
np.argmax(abs(lambda_))*j*2
```

As we need to find the corresponding eigenvector for the largest eigenvalue. Since "the eigenvalues are not necessarily ordered" according to the specs

<https://numpy.org/doc/stable/reference/generated/numpy.linalg.eig.html> and `v[:,i]` is the eigenvector corresponding to the eigenvalue `lambda_[i]`; we should find the correct column of the eigenvector by `np.argmax(abs(lambda_))`.

Share Edit Follow Flag

edited Jan 25 at 10:47

answered Jan 25 at 9:00



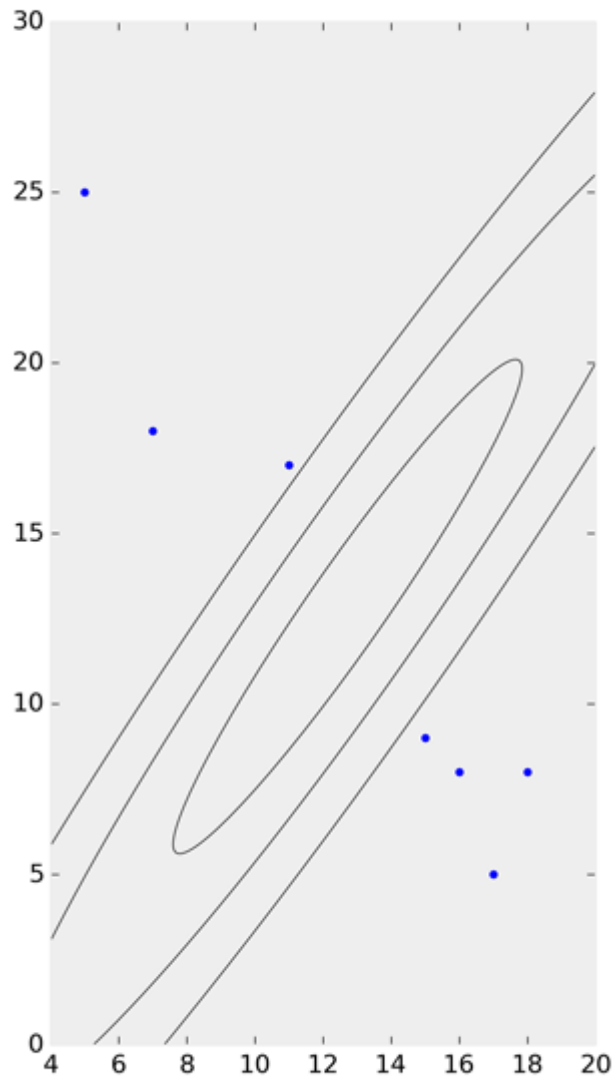
[Kerem](#)

1 2

▲ After giving the accepted answer a go, I found that it doesn't choose the quadrant correctly when calculating theta, as it relies on [np.arccos](#):

26





Taking a look at the [possible duplicate](#) and [Joe Kington's solution on github](#), I watered his code down to this:

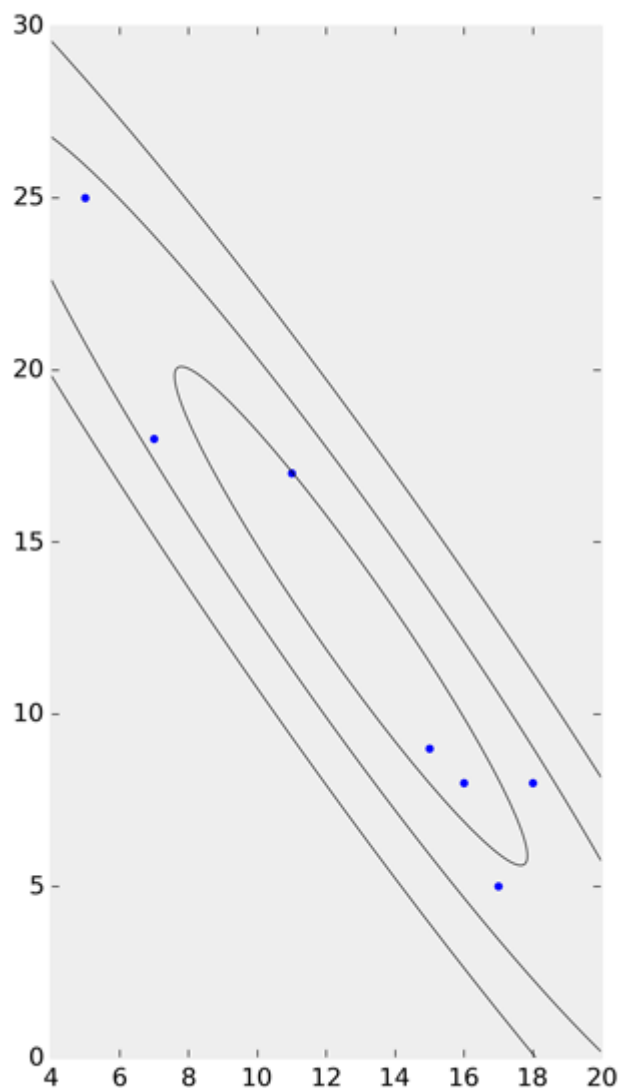
```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Ellipse

def eigsorted(cov):
    vals, vecs = np.linalg.eigh(cov)
    order = vals.argsort()[::-1]
    return vals[order], vecs[:,order]

x = [5, 7, 11, 15, 16, 17, 18]
y = [25, 18, 17, 9, 8, 5, 8]

nstd = 2
ax = plt.subplot(111)

cov = np.cov(x, y)
vals, vecs = eigsorted(cov)
theta = np.degrees(np.arctan2(*vecs[:,0][::-1]))
w, h = 2 * nstd * np.sqrt(vals)
ell = Ellipse(xy=(np.mean(x), np.mean(y)),
              width=w, height=h,
              angle=theta, color='black')
ell.set_facecolor('none')
ax.add_artist(ell)
plt.scatter(x, y)
plt.show()
```



Share Edit Follow Flag

edited May 23 '17 at 12:09

answered Jul 29 '14 at 18:38



Community ♦

1 1



Ben

403 4 6

2 ▲ Does anyone know how to generalise this to 3D (or possibly n dimensions)? – [HansSnah](#) Aug 31 '18 at 19:50



▲ @Ben Why $w, h = 2 * nstd * np.sqrt(vals)$? That's $4 * np.sqrt(vals)$. – [tauran](#) Feb 20 '19 at 17:12



▲ why it doesn't work when I change `plt.scatter(x, y)` to `plt.scatter(np.mean(x), np.mean(y))`. I want the ellipsoid around the mean – [Farshid Rayhan](#) May 28 '20 at 11:58

